Beyond Benchmarks: Evaluating Agentic Models in Production Systems

Anonymous Authors
Anonymous Institution
anonymous@anonymous.org

***Abstract*—Large language model agents are rapidly evolving from code assistants into autonomous software engineering systems, yet evaluation methodologies remain rooted in static, isolated, short-horizon benchmarks that fail to capture the dynamic complexity of real-world production workflows. We present EVA (Evaluating Agentic Models in Production Systems), a production-grounded evaluation framework built upon the YATCC integrated platform. EVA provides a unified evaluation architecture supporting heterogeneous LLM providers and agent SDKs through standardized API access, realistic long-horizon compiler construction workloads with strict serial dependencies, a RESURRECTION mechanism that decomposes cascading failure into diagnostic components, and utility-oriented multi-dimensional metrics measuring both outcome quality and process efficiency. Across 22 model configurations, four agent backends, and two workload difficulty levels, we demonstrate that serial evaluation reveals failure patterns invisible to static benchmarks, that resurrection recovers substantial downstream diagnostic signal (median 73.65-point pipeline score recovery), and that process cost is decoupled from outcome ($R^2 = 0.007$), providing diagnostic rather than predictive value. EVA thus establishes a new paradigm for agent evaluation that moves beyond synthetic benchmarks toward continuous, runtime-observable, production-grounded assessment.**

***Index Terms*—agent evaluation, production systems, compiler construction, serial dependency, resurrection mechanism**

I. INTRODUCTION

Large language models have rapidly evolved from conversational assistants into autonomous software engineering agents capable of planning, implementing, debugging, and deploying complex infrastructure stacks [1]–[3]. Modern agentic development frameworks demonstrate increasingly sophisticated capabilities in repository-level code generation, multi-file reasoning, and tool orchestration. Yet the evaluation methodologies for these systems remain rooted in a *benchmark-driven paradigm*: isolated, static, short-horizon tasks where each evaluation unit is independent and the agent is reset after every attempt.

This paradigm has a fundamental limitation: *real production engineering is not a collection of independent tasks*. In practice, production infrastructure engineering involves long-horizon iterative development, multi-file and cross-module reasoning, dependency management, runtime debugging, tool orchestration, deployment validation, continuous integration pipelines, environment instability, resource constraints, and recovery from cascading failures. Static benchmarks cannot capture these dynamics—they measure instantaneous coding ability, not engineering evolution.

The gap. As a result, strong benchmark performance does not necessarily imply practical engineering utility. Models optimized for benchmark-style coding may still fail in realistic production environments where runtime correctness, execution robustness, and adaptive recovery become critical. We argue that the field needs to move *beyond benchmarks* toward *evaluation frameworks*—systems that assess agentic models in realistic, multi-stage production environments with controlled intermediate state injection, multi-dimensional metrics, and systematic failure diagnosis.

Our approach. We present EVA (Evaluating Agentic Models in Production Systems), a production-grounded evaluation framework built upon the YATCC integrated platform. EVA provides a unified evaluation architecture supporting heterogeneous LLM providers and agent SDKs (OpenHands, Claude Code, Codex CLI, Kimi CLI) through standardized API access via AIHUB. It features realistic long-horizon compiler construction workloads organized as serial dependency chains with two difficulty levels (YATCC and YATCC-Hard), a RESURRECTION mechanism that decomposes cascading failure into diagnostic components, and utility-oriented multi-dimensional metrics measuring both outcome quality and process efficiency.

Our contributions are:

1) We propose EVA as a production-grounded evaluation *framework* (not a benchmark) that moves beyond synthetic, isolated task evaluation toward persistent, runtime-observable, production-grounded assessment of agentic models.
2) We design realistic long-horizon engineering workloads with strict serial dependencies and verifiable intermediate states, spanning two difficulty levels that quantify the value of scaffolding in agentic coding tasks.
3) We introduce a unified evaluation architecture and utility-oriented metrics that support heterogeneous LLMs and agent frameworks, measuring not just outcome scores but process cost, failure patterns, and deployment-relevant efficiency.
4) We provide empirical evidence across 22 model configurations, four agent backends, and two workload difficulty levels, demonstrating that serial evaluation reveals failure patterns invisible to static benchmarks and that resurrection recovers substantial downstream diagnostic signal.

II. Framework Design

A. Design Motivation

Existing agent benchmarks suffer from three fundamental limitations: (1) *task isolation*—each task is independent, preventing measurement of cross-task dependency effects; (2) *outcome-only evaluation*—only final scores are reported, with no visibility into process cost, failure patterns, or recovery behavior; (3) *infrastructure coupling*—each benchmark requires custom evaluation infrastructure, making cross-benchmark comparison and model-backend matrix evaluation impractical.

EVA addresses these limitations through a unified evaluation architecture that decouples workload definition from execution infrastructure, supports heterogeneous model providers and agent frameworks through a standardized API layer, and records both outcome and process metrics as first-class analytical objects.

B. Unified Evaluation Framework

EVA's evaluation framework consists of five major components (Figure **??**):

Task Orchestrator. Responsible for workload scheduling, dependency resolution, resurrection triggering, and evaluation pipeline management. The orchestrator maintains the serial task chain state, monitors per-task pass/fail status, and triggers resurrection when a task falls below the 60% passing threshold.

Agent Runtime Layer. Supports multiple agent frameworks through a unified interface, each representing a distinct interaction paradigm:

OpenHands SDK: Full agent framework with unlimited self-loop iteration, skill injection, and MCP tool integration. Claude Code: File I/O and command execution agent with multi-turn conversation. Codex CLI: Autonomous coding agent with sandboxed execution. Kimi CLI: Conversational coding agent with plan management and background task execution.

All backends share a common task context format and produce standardized result structures, enabling direct cross-backend comparison.

Model Access Layer (AIHUB). A unified API gateway (https://aihub.arcsysu.cn) providing standardized OpenAI-compatible access to 21 large language models spanning proprietary and open-weight families. This design eliminates API-specific integration overhead and ensures that evaluation differences reflect model and scaffold capabilities rather than infrastructure variability.

Execution Environment. Each evaluation run operates in an isolated workspace containing a complete copy of the target repository with all dependencies (ANTLR, LLVM 14, pybind11) pre-installed. For YatCC-Hard, each run additionally launches a fresh container environment that is destroyed after completion, ensuring zero cross-run contamination. The workspace is configured via CMake with a fixed toolchain; scoring scripts are deterministic.

Observation & Leaderboard Layer. Collects and aggregates evaluation results, computes utility-oriented metrics, and maintains dynamic leaderboards. All raw evaluation data (scores, logs, process metrics) is preserved for reproducibility.

Task	Name	Objective
0	Environment Setup	Build & verify toolcha
1	Lexer	Tokenize C source cod
2	Parser	Construct AST/ASG
3	IR Generation	Emit LLVM IR
4	IR Optimization	Implement LLVM Pass
5	Code Generation	Produce RV64 assemb

TABLE I: EVA task chain. Each task consumes the output artifact of i

C. Workloads: Compiler Construction as Realistic Evaluation

EVA grounds its evaluation in compiler construction, a domain chosen for its strict causal dependencies, well-defined intermediate artifacts, and objective automated scoring. Compiler engineering provides an ideal evaluation substrate because: (1) the pipeline has *strict causal dependencies*—each stage consumes the output of its predecessor; (2) intermediate artifacts are *well-defined and verifiable*—token streams, parse trees, IR modules, and assembly files have clear structural specifications; (3) the project includes *automated scoring scripts* with comprehensive test suites.

We provide two complementary workloads spanning a difficulty spectrum:

1) YatCC: Scaffolded Implementation: The original YatCC workload [4] organizes six tasks along the compiler construction pipeline (Table I). Each task provides *scaffolded starter code*—the agent fills in missing implementation details within a provided framework. This workload measures an agent's ability to *complete* a partially-implemented system.

2) YatCC-Hard: Full From-Scratch Implementation: YatCC-Hard removes all code logic from the original tasks, retaining only file dependency relationships to ensure automated scripts remain functional. Agents receive *no compiler construction knowledge* and *no implementation framework*—they must implement the full compiler from scratch. The task descriptions are reduced to experiment requirements and scoring script usage instructions. This workload measures an agent's ability to *construct* a complete system from minimal specification.

The difficulty gap between YatCC and YatCC-Hard quantifies the value of scaffolding in agentic coding tasks and reveals which models depend on provided structure versus which can operate autonomously.

Importantly, these workloads have been validated through years of practical use in compiler construction education at Sun Yat-sen University, ensuring their authenticity as real engineering tasks rather than synthetic benchmarks.

D. Resurrection Protocol

The Resurrection mechanism addresses the fundamental evaluation challenge of serial workloads: when an agent fails at Task k, downstream tasks become unreachable, and the benchmark can only report "failure at k" without diagnosing whether the agent could succeed at Tasks $k+1, \ldots, 5$ given correct prerequisites.

Definition. When Task k scores below the passing threshold (60%), Resurrection is triggered: (1) the agent's failed

artifact is replaced with the *golden intermediate artifact*—the correct output produced by the reference implementation; (2) the build system is reconfigured via `config.cmake` (`TASK_k_REVIVE=ON`); (3) the agent continues to Task $k+1$ with the corrected state.

Design rationale. Resurrection is an *evaluation protocol*, not a scoring adjustment. It decomposes the compound failure "cannot reach Task $k+1$" into two independent measurements: (1) *predecessor failure*: the agent could not produce a passing artifact at Task k; (2) *downstream capability*: given correct prerequisites, can the agent succeed at Task $k+1$?

This decomposition enables node-level pass rate reporting even when the full pipeline is broken, significantly improving diagnostic resolution. Agents are *not informed* about resurrection—the mechanism is triggered externally by the evaluation orchestrator, eliminating moral hazard.

E. Intermediate Artifacts and Verification

Each task produces a well-defined intermediate artifact that serves as both the evaluation object for the current task and the input dependency for the next: Task 1 produces a *token stream*; Task 2 produces an *AST/ASG*; Task 3 produces *LLVM IR*; Task 4 produces *optimized IR*; Task 5 produces *RV64 assembly*. Scoring is performed by dedicated test suites that execute the agent's code against comprehensive test cases. Each task's score ranges from 0 to 100, with a passing threshold of 60%.

III. Metrics & Experimental Setup

A. Utility-Oriented Multi-Dimensional Metrics

EVA proposes a multi-dimensional evaluation metric system that moves beyond simple accuracy scores toward utility-oriented measurement. Evaluation metrics should measure not just *whether* an agent succeeds, but *how efficiently* it succeeds and *how* it fails.

Outcome metrics. We define three primary outcome metrics. *Mean Reward (MR)* is the weighted average of task scores with resurrection bonus: $\text{MR} = \sum s_i w_i b_i / \sum w_i b_i$, where $w_i = [0.05, 0.20, 0.20, 0.15, 0.30, 0.10]$ and $b_i = 1.2$ (no resurrection) or 1.0 (resurrected). *Pass Score (PS)* is the weighted pass rate: $\text{PS} = \sum p_i b_i^{\text{pass}} / (6 \times 1.5) \times 100$, where $p_i = 1$ if $s_i \geq 60$ and $b_i^{\text{pass}} = 1.5$ (no resurrection) or 1.0. *Pipeline Score (PL)* is the simple average: $\text{PL} = \frac{1}{6} \sum s_i$.

Process metrics. Beyond outcome scores, we record token consumption, interaction turns, command executions, retry counts, and elapsed wall-clock time per task and total.

Failure taxonomy. We categorize agent failures into five types: (1) *predecessor failure*—downstream unreachable due to missing artifact; (2) *task-local reasoning failure*—cannot solve despite correct prerequisites; (3) *execution failure*—build/dependency errors; (4) *process collapse*—max-turn exhaustion, search loops, repetitive debugging without progress; (5) *cost overrun*—budget exhausted.

This taxonomy enables systematic analysis of *how* agents fail, not just *whether* they fail. For instance, process collapse is more prevalent in framework-based agents with unlimited iteration budgets, suggesting that scaffold design affects failure mode distribution.

Backend	Type	Key Proper
OpenHands SDK	Full agent framework	Unlimited se
Codex CLI	CLI agent	Autonomous
Claude Code	CLI agent	File I/O, cor
Kimi CLI	CLI agent	Conversation

TABLE II: Agent backends used in E

B. Models and Agent Backends

We evaluate EVA across 22 model configurations spanning both proprietary and open-weight LLMs, using four agent backends:

All models are accessed through the AIHUB unified API gateway, ensuring consistent infrastructure conditions across evaluations.

C. Evaluation Settings

We conduct evaluation under two primary settings:

Setting 1: Serial pipeline with resurrection (default). Agents proceed through Tasks 0–5 in sequence. When a task fails, resurrection is triggered and the agent continues with the golden intermediate artifact. This setting produces the full leaderboard with both outcome and process metrics.

Setting 2: Serial pipeline without resurrection. Agents proceed through Tasks 0–5 in sequence, but *no resurrection is triggered*. When a task fails, all subsequent tasks receive the agent's (possibly broken) artifact and typically cascade to zero scores. This setting measures *zero-shot pipeline depth*—how far an agent can progress without any external correction.

The comparison between Setting 1 and Setting 2 directly tests whether resurrection recovers diagnostic signal lost to cascading failure.

D. Workload Comparison: YatCC vs. YatCC-Hard

We evaluate models on both workloads to quantify the difficulty gap between scaffolded and from-scratch implementation. YatCC provides starter code and implementation frameworks; YatCC-Hard removes all code logic, requiring agents to construct the full compiler from minimal specification. The performance delta between the two workloads measures the value of scaffolding and reveals which models depend on provided structure versus which can operate autonomously.

Experiments are conducted under unified infrastructure conditions using identical execution environments and workload distributions, ensuring that observed differences reflect genuine model and scaffold capability gaps rather than environmental variability.

IV. Results & Analysis

A. Main Leaderboard

Table III presents the full EVA leaderboard across 22 model configurations on the YatCC workload.

Model	Backend	T0	T1	T2	T3	T4	T5	MR	PS	PL	Res
kimi-k2.6	kimi	**100**	**100**	**100**	98.6	97.4	**100**	99.1	77.8	99.3	4
kimi-k2.5	kimi	**100**	**100**	**100**	98.6	95.5	**100**	98.5	77.8	99.0	4
gpt-5.5	codex	**100**	**100**	**100**	**100**	**95**	**100**	98.5	100.0	99.2	0
claude-sonnet-4.6	claude	**100**	**100**	**100**	98.6	94.6	**100**	98.3	77.8	98.9	4
mimo-v2.5-pro	kimi	**100**	**100**	**100**	98.6	94.5	**100**	98.2	77.8	98.9	4
qwen3.6-flash	openhands	**100**	**100**	**100**	**100**	**92**	**100**	97.5	66.7	98.6	2
glm-5.1	openhands	**100**	**100**	**100**	**100**	91.1	**100**	97.3	100.0	98.5	0
deepseek-v4-pro	openhands	**100**	**100**	**100**	**100**	71	**100**	91.4	66.7	95.2	0
minimax-m2.7	openhands	**100**	61	**100**	**93**	**98**	**100**	90.4	66.7	91.9	2
qwen3.6-max-preview	openhands	**100**	61	**100**	**97**	**94**	**100**	90.0	66.7	92.0	3
glm-5	openhands	**100**	**100**	**100**	**100**	63	**100**	88.8	66.7	93.8	1
qwen3.6-plus	openhands	**100**	**100**	74	73	**81**	**100**	85.0	66.7	88.0	2
deepseek-v4-flash	openhands	**100**	0	**100**	**93**	**90**	**100**	75.8	55.6	80.5	3
glm-4.7	openhands	**100**	0	62	**100**	**94**	**100**	70.5	55.6	75.9	1
minimax-m2.5	openhands	**100**	**100**	0	73	**80**	**100**	70.0	55.6	75.5	2
qwen3.5-plus	openhands	**100**	**100**	0	53	**87**	**100**	69.0	44.4	73.3	2
qwen3-coder-flash	openhands	**100**	**100**	0	34	58	**100**	67.5	33.3	73.7	3
mimo-v2.5-pro	openhands	**100**	0	**100**	**100**	60	**100**	67.0	44.4	70.8	3
deepseek-chat	openhands	**100**	0	21	**100**	60	**100**	52.2	44.4	63.5	2
deepseek-reasoner	openhands	**100**	0	21	**100**	60	**100**	52.2	44.4	63.5	2
kimi-k2-thinking	openhands	**100**	0	21	**100**	60	**100**	52.2	44.4	63.5	2
qwen-omni-turbo	openhands	**100**	0	0	0	0	0	50.9	33.3	62.6	2

TABLE III: EVA YATCC leaderboard across 22 model configurations. Underlined models achieved zero-shot pipeline success (no resurrection). **Bold** scores indicate task passing ($\geq 60\%$). MR = Mean Reward, PS = Pass Score, PL = Pipeline Score, Res = Resurrection count.

Key observation. Only 3 out of 22 configurations (GPT-5.5/Codex, deepseek-v4-pro/OpenHands, and glm-5.1/OpenHands) achieve zero-shot pipeline success without any resurrection. The top 5 positions are dominated by Kimi CLI backend configurations (kimi-k2.6, kimi-k2.5, mimo-v2.5-pro), demonstrating that scaffold choice significantly impacts outcomes—the same model `mimo-v2.5-pro` scores 98.2 on Kimi CLI versus 67.0 on OpenHands, a 31.2-point gap.

B. Resurrection Recovers Hidden Downstream Capability

Figure 1 shows the per-task score improvement when resurrection is enabled versus the no-resurrection setting. Without resurrection, 19 out of 22 configurations cannot reach the full pipeline depth of 6 tasks; the median zero-shot pipeline depth is only 1 task. With resurrection, all configurations can be evaluated at every downstream node, recovering diagnostic signal that would otherwise be lost.

The most striking example is `claude-sonnet-4.6`: without resurrection, it scores 0 on Tasks 3–5 (pipeline depth = 3, PL = 50.0), but after resurrection at Tasks 2–5, achieves 98.6 on Task 3, 94.6 on Task 4, and 100 on Task 5 (PL = 98.9, MR = 98.3, Rank 4). This represents a *48.9-point* pipeline score improvement—demonstrating that the agent's failure at Task 2 was purely a predecessor effect, not downstream incapability.

Systematic analysis. The resurrection gain is not limited to cherry-picked examples: 17 out of 19 non-zero-shot configurations achieve $>$ 40-point pipeline score improvement with resurrection, and the median gain is 73.65 points. Only the three zero-shot configurations show zero gain. The mean resurrection gain across all configurations is 71.55 points, demonstrating that cascading failure systematically obscures downstream capability in serial workloads—resurrection recovers this lost signal at scale.

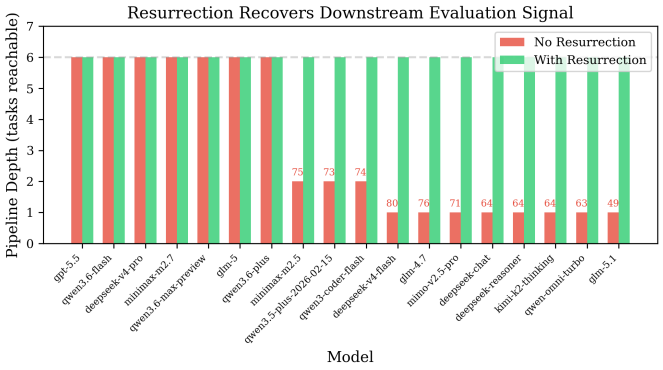

Fig. 1: Resurrection recovers downstream evaluation signal. Without resurrection, most agents can only be evaluated at 1–2 tasks before cascading failure blocks further measurement. With resurrection, all 6 tasks become reachable, revealing hidden downstream capability.

C. Serial Evaluation Preserves Overall Ranking but Reveals Specific Failure Patterns

To test whether serial evaluation provides different information than static evaluation, we define an *independent-task baseline*: for each model, we compute the average of per-task scores obtained *with resurrection* (i.e., with golden prerequisites provided at each task), which measures each task's capability in isolation. A Kendall τ correlation of 0.763 ($p < 0.001$) between

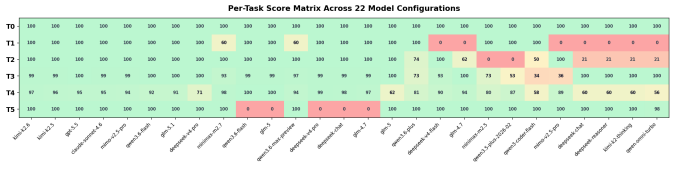

Fig. 2: Per-task score matrix across 22 configurations. The heatmap reveals distinct failure patterns: some models fail at early tasks (T1/T2) while others fail at later tasks (T4), demonstrating that serial evaluation captures qualitatively different information than independent-task benchmarks.

Model	T0	T1	T2	T3
qwen3.6-max-preview	100	100	86.0	86.3
deepseek-v4-pro	100	100	79.5	87.7
glm-5.1	100	100	89.1	91.8
deepseek-v4-flash	100	100	77.8	89.0
qwen3.6-plus	100	100	77.1	19.2
glm-5	100	100	65.7	65.8
minimax-m2.7	100	80.4	33.5	41.1
kimi-k2.6	100	100	18.0	41.1

TABLE IV: YATCC-Hard leaderboard (top 8 of 18). Scores drop significa only PL=94.5 on Hard vs. PL=99.3 on YATCC.

the no-resurrection serial ranking and the independent-task ranking confirms that the two rankings are *strongly correlated*—serial evaluation does not wholesale reorder the leaderboard. However, $\tau = 0.763$ is not 1.0: specific models experience large directional shifts invisible in aggregate correlation statistics.

The most dramatic rank shifts occur for models that fail at early tasks but demonstrate strong downstream capability: `deepseek-v4-flash` drops from independent-task rank 8 to no-resurrection rank 14 (shift of 6 positions), because its Task 1 failure blocks the entire pipeline despite achieving 93.2% on Task 3 and 89.5% on Task 4 with resurrection. A Wilcoxon signed-rank test on the paired rank differences yields $W = 34.5$, $p = 0.44$, indicating that the *overall* ranking structure is preserved, but individual models can experience large directional shifts. The key finding is that serial evaluation *selectively penalizes* models with early-stage failures while *rewarding* those with consistent cross-task continuity.

D. Model–Scaffold Interaction

The choice of agent scaffold significantly affects outcomes. The same model (`mimo-v2.5-pro`) scores 67.0 on OpenHands versus 98.2 on Kimi CLI—a 31.2-point gap attributable entirely to scaffold design differences in action space, retry logic, and context management. The top 5 positions are occupied by CLI-based backends (Kimi CLI: 3, Codex: 1, Claude Code: 1), while OpenHands-based configurations occupy ranks 6 and below. This confirms that *scaffold design is an independent variable* that must be controlled and reported in evaluation frameworks.

E. YATCC *vs.* YATCC-*Hard: The Scaffolding Gap*

Table IV presents the YATCC-Hard leaderboard across 18 model configurations. The difficulty gap between the two workloads quantifies the value of scaffolding in agentic coding tasks.

Key observation. The best YATCC-Hard score (PL=94.5 by `qwen3.6-max-preview`) is 4.8 points below the best YATCC score (PL=99.3 by `kimi-k2.6`). More dramatically, models that excel on YATCC often collapse on YATCC-Hard: `kimi-k2.6` drops from PL=99.3 (Rank 1 on YATCC) to PL=64.9 (Rank 8 on YATCC-Hard), a 34.4-point decline. Similarly, `mimo-v2.5-pro` drops from PL=98.9 (Rank 5 on YATCC) to PL=0.0 on YATCC-Hard—a complete collapse. This gap reveals that scaffolded starter code provides substantial leverage, and that models appearing capable when filling in blanks may lack the architectural reasoning required for full from-scratch implementation.

The resurrection mechanism is equally critical on YATCC-Hard: 12 out of 18 configurations require at least one resurrection. This confirms that cascading failure is a workload-independent phenomenon—it affects both scaffolded and from-scratch settings, and resurrection provides diagnostic value in both contexts. Notably, 6 out of 18 models score 0.0 on YATCC-Hard (including `mimo-v2.5-pro`, `kimi-k2-thinking`, and `deepseek-chat`), indicating a fundamental capability threshold that current models cannot cross without scaffolding.

F. Cost-Performance Frontier

Figure 3 plots total elapsed time against pipeline score for all evaluated configurations. Two patterns emerge: (1) *high-score configurations are not always low-cost*—deepseek-reasoner achieves PL=63.5 but requires 15,987 seconds and 252 build attempts, while qwen3.6-flash achieves PL=98.6 in only 1,421 seconds with 52 builds; (2) *cost efficiency varies dramatically*—the same pipeline score of 63.5 is achieved by three models with elapsed times ranging from 1,909s to 15,987s (an 8x difference).

Process cost does not predict outcome. A linear regression of build count against pipeline score yields $R^2 = 0.003$ ($p = 0.82$); elapsed time against pipeline score yields $R^2 < 0.001$ ($p = 0.98$). The combined model explains only 0.7% of score variance. This *decoupling* is itself a diagnostic finding: agents that invest more computational budget (time, builds, retries) do not systematically achieve better outcomes. The relationship is not monotonic—some agents achieve high scores with minimal effort (efficient solvers), while others exhaust their budget without progress (process collapse). This demonstrates that process metrics capture *qualitatively distinct* information from outcome metrics—not because they predict scores, but because they reveal *how* agents arrive at their outcomes. Two agents with identical pipeline scores may have radically different process profiles (efficient solver vs. persistent searcher), and this distinction is invisible to outcome-only evaluation but critical for deployment decisions (cost budgeting, timeout setting, scaffold optimization). Process metrics thus serve a *diagnostic* rather than *predictive* role: they do not explain *why* an agent succeeds, but they reveal *how* it attempts to succeed,

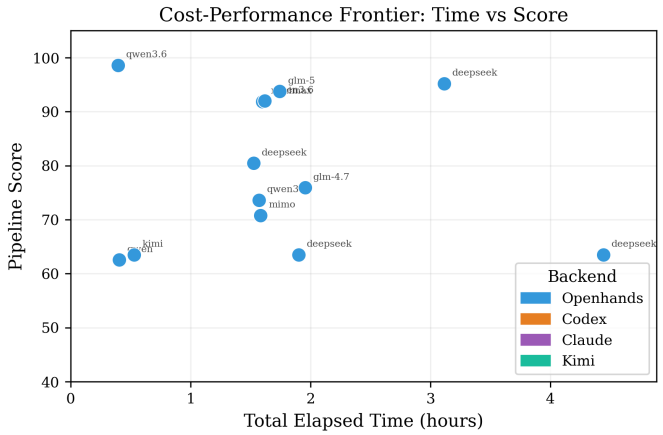

Fig. 3: Cost-performance frontier: elapsed time vs. pipeline score. The Pareto frontier highlights configurations achieving the best performance per unit time. The 8x cost difference between similarly-scoring agents (kimi-k2-thinking at 1,909s vs. deepseek-reasoner at 15,987s, both PL≈63.5) demonstrates that outcome-only evaluation misses critical deployment information.

enabling targeted intervention when the process profile indicates inefficiency or collapse.

Build attempt counts reveal distinct agent behaviors: *efficient solvers* (GPT-5.5: 80 builds, qwen3.6-flash: 52 builds) achieve high scores with minimal retries; *persistent searchers* (deepseek-v4-flash: 181 builds, deepseek-reasoner: 252 builds) achieve moderate scores through exhaustive search; and *collapsed agents* exhaust builds without progress. Agents that repeatedly build without modifying code exhibit *process collapse*—a failure mode where the action loop decouples from productive problem-solving.

G. Failure Taxonomy

We categorize agent failures into five types: (1) *predecessor failure* (downstream unreachable), (2) *task-local reasoning failure* (cannot solve despite correct prerequisites), (3) *execution failure* (build/dependency errors), (4) *process collapse* (max-turn exhaustion, search loops), and (5) *cost overrun* (budget exhausted).

Figure 4 shows that predecessor failure dominates in the no-resurrection setting but is largely eliminated by resurrection, revealing the underlying distribution of task-local and execution failures. Process collapse is more prevalent in framework-based agents with unlimited iteration budgets, suggesting that scaffold design affects not just *whether* the agent succeeds, but *how* it fails.

V. Discussion

A. Why Production-Grounded Evaluation Matters

Our results demonstrate that serial dependency introduces an evaluation axis invisible to static benchmarks. When tasks are independent, an agent's failure at one task does not affect others,

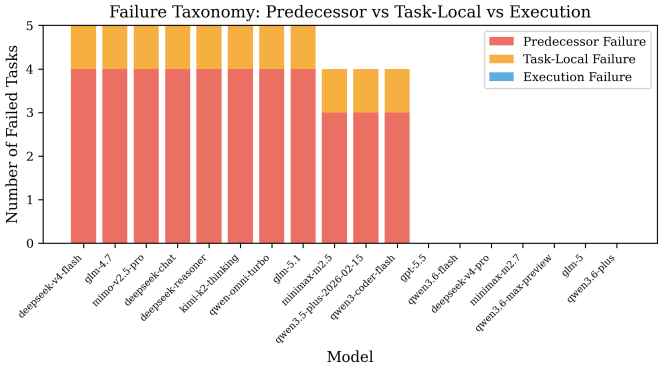

Fig. 4: Failure taxonomy composition. Predecessor failure dominates without resurrection but is largely eliminated by resurrection, revealing the underlying task-local and execution failure distribution.

and the benchmark can measure each capability in isolation. When tasks are serially dependent, failure propagates, and the evaluation framework must distinguish between "cannot reach" and "cannot solve"—a distinction that resurrection enables.

This is not merely a technical detail but a fundamental shift in what evaluation frameworks measure: from *can the agent solve task k?* to *can the agent progress along the dependency chain, and if not, where and why?* Our finding that only 3 out of 22 configurations achieve zero-shot pipeline success—despite most models scoring highly on individual tasks—demonstrates that static benchmarks systematically overestimate production readiness.

B. Resurrection as Methodology, Not Assistance

A natural concern is that resurrection "helps" the agent by providing correct intermediate artifacts, thereby inflating scores. We emphasize that resurrection is an *evaluation protocol*, not a scoring adjustment. The no-resurrection setting provides the "strict" measurement of zero-shot pipeline capability. The resurrection setting provides the "diagnostic" measurement of node-level capability given correct prerequisites. Both are reported independently; neither replaces the other.

Potential biases. We acknowledge three potential biases: (1) *state discontinuity*—the golden artifact may differ in style from the agent's own output; (2) *selection bias*—only failed tasks are resurrected; (3) *moral hazard*—if agents were aware of resurrection, they might reduce early-stage effort. We mitigate bias (3) by ensuring agents are *not informed* about resurrection—the mechanism is triggered externally by the evaluation orchestrator.

C. Process Cost as Diagnostic Signal

Our regression analysis reveals that process cost (build count, elapsed time) explains essentially zero percent of outcome variance ($R^2 = 0.007$). This is not a failure of process metrics—it is a finding with direct implications for production deployment. Agents that invest more computational budget do not systematically achieve better outcomes. Two agents

with identical pipeline scores may have radically different process profiles (efficient solver vs. persistent searcher), and this distinction is invisible to outcome-only evaluation but critical for deployment decisions regarding cost budgeting, timeout setting, and scaffold optimization.

D. The Scaffolding Gap

The performance delta between YATCC and YATCC-Hard quantifies the value of scaffolding in agentic coding tasks. Top models on YATCC (e.g., `kimi-k2.6` at PL=99.3) drop to PL=64.9 on YATCC-Hard—a 34.4-point decline. This reveals that scaffolded starter code provides substantial leverage, and that models appearing capable when filling in blanks may lack the architectural reasoning required for full from-scratch implementation. The scaffolding gap thus serves as a diagnostic metric for distinguishing between *completion capability* and *construction capability*.

E. Limitations

Domain specificity. EVA is grounded in compiler construction, a domain with particularly clean intermediate artifact specifications. Other serial workflows—ML pipelines, DevOps chains, data engineering flows—may have less well-defined intermediate states, making resurrection harder to implement. We do not claim that EVA generalizes to all serial settings, but argue that the *evaluation methodology* (serial dependency + resurrection + process metrics) is transferable.

Task count and statistical power. With six tasks, EVA provides limited statistical power for per-task analysis. However, the serial structure means each task carries *qualitatively different* information (different dependency depth, different failure propagation pattern), which compensates for the small task count with structural diversity. Each model configuration is evaluated once per pipeline run; multi-seed evaluation is planned as future work.

Contamination risk. YATCC is a public GitHub repository, and frontier models may have been trained on its contents. However, EVA's key challenge is not reproducing known solutions but maintaining *serial continuity*—producing correct intermediate artifacts that integrate across the pipeline. Memorizing individual task solutions does not guarantee correct cross-task artifact flow. Future versions could adopt temporal splitting or private codebases to mitigate this concern.

F. Future Directions

Multi-domain workloads: extending the evaluation methodology to ML engineering, DevOps, and data science pipelines. Evolutionary gain measurement: comparing agent performance when inheriting self-produced vs. golden intermediate artifacts, quantifying how well agents "understand their own code." Dynamic resurrection: allowing agents to request resurrection proactively, studying self-diagnosis capability. Expanded model and scaffold coverage: evaluating additional frontier models and agent frameworks as they become available. Infrastructure-aware workloads: incorporating deployment validation, runtime monitoring, and recovery scenarios.

VI. Related Work

Repository-level coding benchmarks. SWE-bench [1] established the paradigm of evaluating agents on real GitHub issues with execution-based patch verification, demonstrating that repository-level context and test suites provide more realistic assessment than isolated function synthesis [5]. SWE-bench Verified [6] refined this with human-validated task specifications, and subsequent extensions have broadened coverage to multilingual settings [7], longer-horizon tasks [8], mobile development [9], and hardware engineering [10]. FEA-Bench [11] extended the paradigm to feature implementation rather than bug fixing. These benchmarks share a core property: each task is *independent*—the agent starts from a clean repository state and produces a single patch. EVA inherits the execution-grounded verification tradition but fundamentally differs by requiring agents to progress along a *serial dependency chain* where intermediate artifacts accumulate across tasks.

Interactive agent benchmarks. AgentBench [2] evaluates LLM agents across eight interactive environments spanning OS operations, databases, and web tasks. GAIA [12] tests general assistant competence on multi-step reasoning questions requiring tool use. OSWorld [13] and WebArena [14] assess GUI and web navigation in real computer environments. Terminal-Bench [15] provides execution-grounded evaluation in command-line settings. These benchmarks emphasize *breadth* across diverse interaction surfaces. EVA instead emphasizes *depth* along a single dependency chain, enabling precise measurement of cascading failure, intermediate artifact continuity, and process cost accumulation that broader suites cannot isolate.

Paired and augmentation-oriented evaluation. SkillsBench [16] introduced the methodology of evaluating every task under both vanilla and skill-augmented conditions, demonstrating that augmentation efficacy is context-dependent and requires paired measurement. SWE-Skills-Bench [17] applied this methodology to software engineering, finding that most public skills provide negligible benefit. EVA adopts the paired evaluation philosophy but applies it to a different axis: *with vs. without resurrection*, measuring how much diagnostic signal is recovered when predecessor failures are bypassed. This parallels SkillsBench's "no-skill vs. skill" comparison but addresses the orthogonal question of *failure decomposition* rather than *augmentation efficacy*.

Long-horizon and self-evolving agent evaluation. Recent work has begun examining agents over extended trajectories. Frontier-Eng [3] benchmarks self-evolving agents on real-world RD workflows. RE-Bench [18] and PaperBench [19] evaluate frontier research engineering tasks. Voyager [20] demonstrated skill accumulation in Minecraft environments. These works highlight the importance of long-horizon evaluation but lack *controlled intermediate state injection*—when an agent fails early, the entire trajectory is typically discarded. EVA's resurrection mechanism provides a principled way to continue evaluation after early failure, enabling measurement of both evolutionary gain (using self-produced artifacts) and latent

Benchmark	Serial	Exec.	Resurr.	Process	Artifact	Tasks
SWE-bench [1]	—	✓	—	—	—	2.3K
AgentBench [2]	—	✓	—	—	—	8 envs
GAIA [12]	—	—	—	—	—	466
OSWorld [13]	—	✓	—	—	—	369
WebArena [14]	—	✓	—	—	—	812
Terminal-Bench [15]	—	✓	—	—	—	50
SkillsBench [16]	—	✓	—	—	—	1.5K
RE-Bench [18]	—	✓	—	partial	—	7
EVA	✓	✓	✓	✓	✓	6

TABLE V: Benchmark comparison. ✓= supported, — = not supported. *Serial* = tasks form a dependency chain; *Exec.* = execution-grounded scoring; *Resurr.* = resurrection mechanism; *Process* = process metrics as first-class output; *Artifact* = intermediate artifact continuity tracked.

downstream capability (using golden artifacts).

Table V positions EVA against existing benchmarks across five evaluation dimensions.

EVA is the only benchmark that combines serial dependency, execution-grounded scoring, resurrection, process metrics, and artifact continuity tracking. While SWE-bench and SkillsBench share execution-grounding and paired evaluation (respectively), neither addresses serial dependency or intermediate artifact flow. RE-Bench records partial process metrics but lacks resurrection and artifact tracking. The combination of these five dimensions enables EVA to measure evaluation axes that no existing benchmark captures simultaneously.

VII. Conclusion

We presented EVA, a production-grounded evaluation framework for assessing agentic models under realistic infrastructure engineering conditions. Unlike traditional benchmarks that provide fixed datasets and scoring functions, EVA provides a *methodology*—a unified evaluation architecture supporting heterogeneous LLM providers and agent SDKs, realistic long-horizon compiler construction workloads with strict serial dependencies, a Resurrection mechanism that decomposes cascading failure into diagnostic components, and utility-oriented multi-dimensional metrics measuring both outcome quality and process efficiency.

By leveraging authentic workloads from the YatCC platform, our framework continuously evaluates frontier models through genuine development tasks spanning lexical analysis, syntax parsing, IR generation, optimization, and backend code generation. Our experiments across 22 model configurations, four agent backends, and two workload difficulty levels demonstrate that: (1) serial evaluation preserves overall rankings ($\tau = 0.763$) but reveals specific failure patterns invisible to static benchmarks; (2) resurrection recovers substantial downstream diagnostic signal (median 73.65-point pipeline score recovery, 17/19 configurations with > 40-point improvement); (3) process cost is decoupled from outcome ($R^2 = 0.007$), providing diagnostic rather than predictive value; (4) the difficulty gap between YatCC and YatCC-Hard (up to 34.4-point decline for top models) quantifies the value of scaffolding and reveals which models depend on provided structure versus which can operate autonomously.

These findings suggest that scaffold design and evaluation methodology may become as important as foundation model capability itself in determining real-world agent performance. EVA thus establishes a new paradigm for agent evaluation that moves beyond synthetic, isolated benchmarks toward continuous, runtime-observable, production-grounded assessment. We release EVA as an open-source framework with complete evaluation infrastructure, scoring scripts, containerized execution environments, and reproducible workflows.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024, oral presentation.

[2] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang *et al.*, "Agentbench: Evaluating llms as agents," in *The Twelfth International Conference on Learning Representations*, 2024.

[3] others, "Frontier-eng: Benchmarking self-evolving agents on real-world r&d workflows," *arXiv preprint*, 2026, arXiv:2604.12290.

[4] S. Y. sen University Compiler Construction Course Team, "Yatcc: Yet another tiny c compiler," https://github.com/arcsysu/YatCC, 2024.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf *et al.*, "Evaluating large language models trained on code," in *International Conference on Learning Representations*, 2021.

[6] N. Chowdhury, J. Aung, C. J. Shern, O. Jaffe, D. Sherburn, G. Starace, E. Mays, R. Dias, M. Aljubeh, M. Glaese, C. E. Jimenez, J. Yang, L. Ho, T. Patwardhan, K. Liu, and A. Madry, "Introducing SWE-bench verified," OpenAI Blog, 2024.

[7] D. Zan *et al.*, "Swe-bench multilingual: Benchmarking issue resolution across 9 languages," *arXiv preprint*, 2025.

[8] Y. Deng *et al.*, "Swe-bench pro: A longer-horizon issue resolution benchmark," *arXiv preprint*, 2025.

[9] P. Yang *et al.*, "Swe-bench mobile: Can llm agents develop industry-level mobile applications?" *arXiv preprint*, 2025.

[10] others, "Hwe-bench: Benchmarking llm agents on real-world hardware engineering," *arXiv preprint*, 2026.

[11] ——, "Fea-bench: A benchmark for evaluating repository-level code feature implementation," in *ACL*, 2025.

[12] G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom, "Gaia: A benchmark for general ai assistants," in *The Twelfth International Conference on Learning Representations*, 2024.

[13] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, Y. Lu, V. Zhong, and T. Yu, "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 52 040–52 094, neurIPS 2024 Datasets and Benchmarks Track.

[14] S. Zhou *et al.*, "Webarena: A realistic web environment for building autonomous agents," in *The Twelfth International Conference on Learning Representations*, 2024.

[15] J. Merrill *et al.*, "Terminal-bench: A benchmark for evaluating llms on terminal tasks," *arXiv preprint*, 2026.

[16] J. Li *et al.*, "Skillsbench: Benchmarking how well agent skills work across diverse tasks," *arXiv preprint*, 2026, arXiv:2602.12670.

[17] others, "Swe-skills-bench: Do agent skills actually help in real-world software engineering?" *arXiv preprint*, 2026, arXiv:2603.15401.

[18] J. Wijk *et al.*, "Re-bench: Benchmarking research engineering agents," *arXiv preprint*, 2024.

[19] G. Starace *et al.*, "Paperbench: Evaluating ai agents on paper reproduction," *arXiv preprint*, 2025.

[20] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Liu, Y. Wang, Y. Zhu, L. Luo, and P. Luo, "Voyager: An open-ended embodied agent with large language models," in *Advances in Neural Information Processing Systems*, 2023.